

汇编语言

15 鲲鹏处理器汇编语言

张青

ieqzhang@zzu.edu.cn

教学内容



- 鲲鹏处理器
- AArch64汇编语言

教材章节：6,7

慕课：11,12,13

- ▶ 鲲鹏处理器
- ▶ AArch64应用层体系结构
 - ▶ 寄存器
 - ▶ 寻址方式
 - ▶ 指令格式
 - ▶ 常用指令
 - ▶ 程序结构
- ▶ 与x86对比

学习目标



- 能说出鲲鹏处理器的特点及与ARM汇编的关系
- 能在鲲鹏架构的华为弹性云服务器将C语言转换为汇编语言
- 能阅读、分析ARM汇编语言
- 能对比x86架构和AArch64架构汇编的区别
- 主动学习，积极探索，勤于思考，善于总结，乐于实践，自主自律

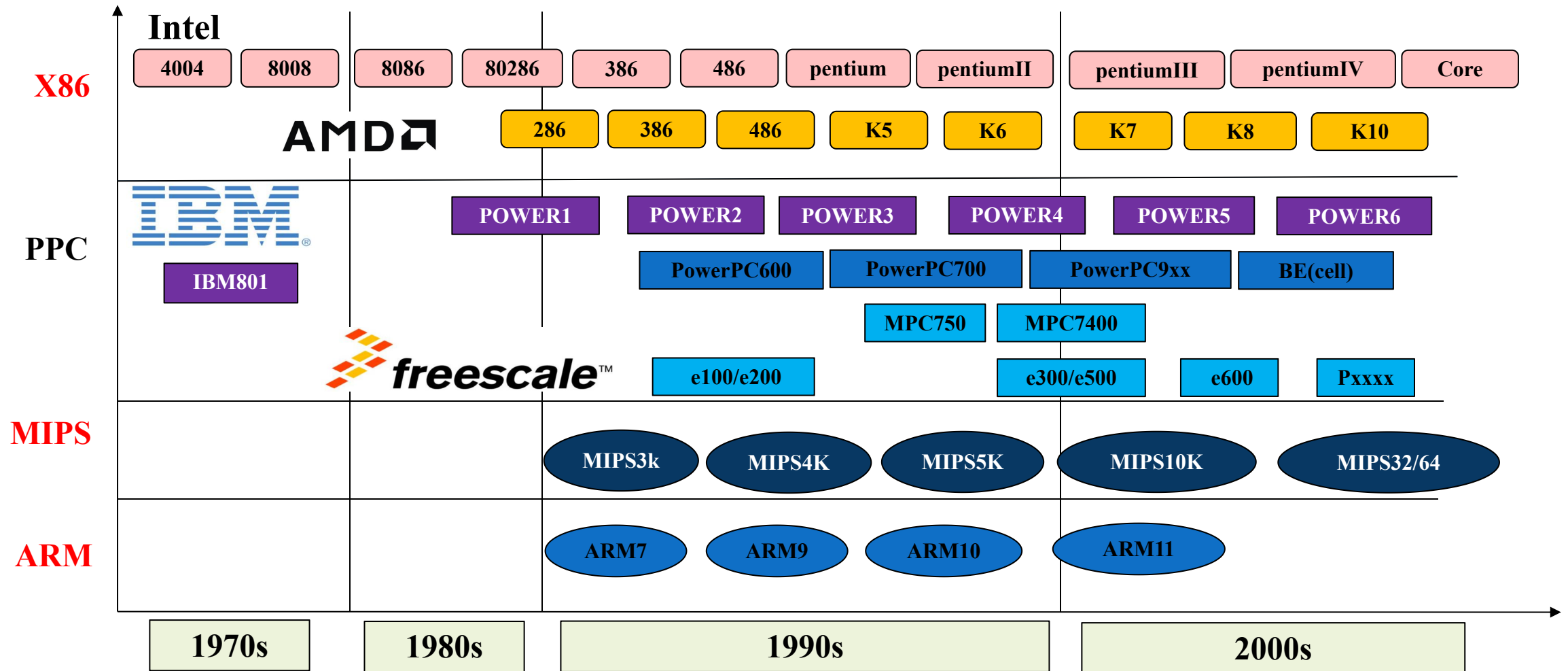
ARM处理器与鲲鹏处理器

中央处理器CPU (Central Processing Unit)

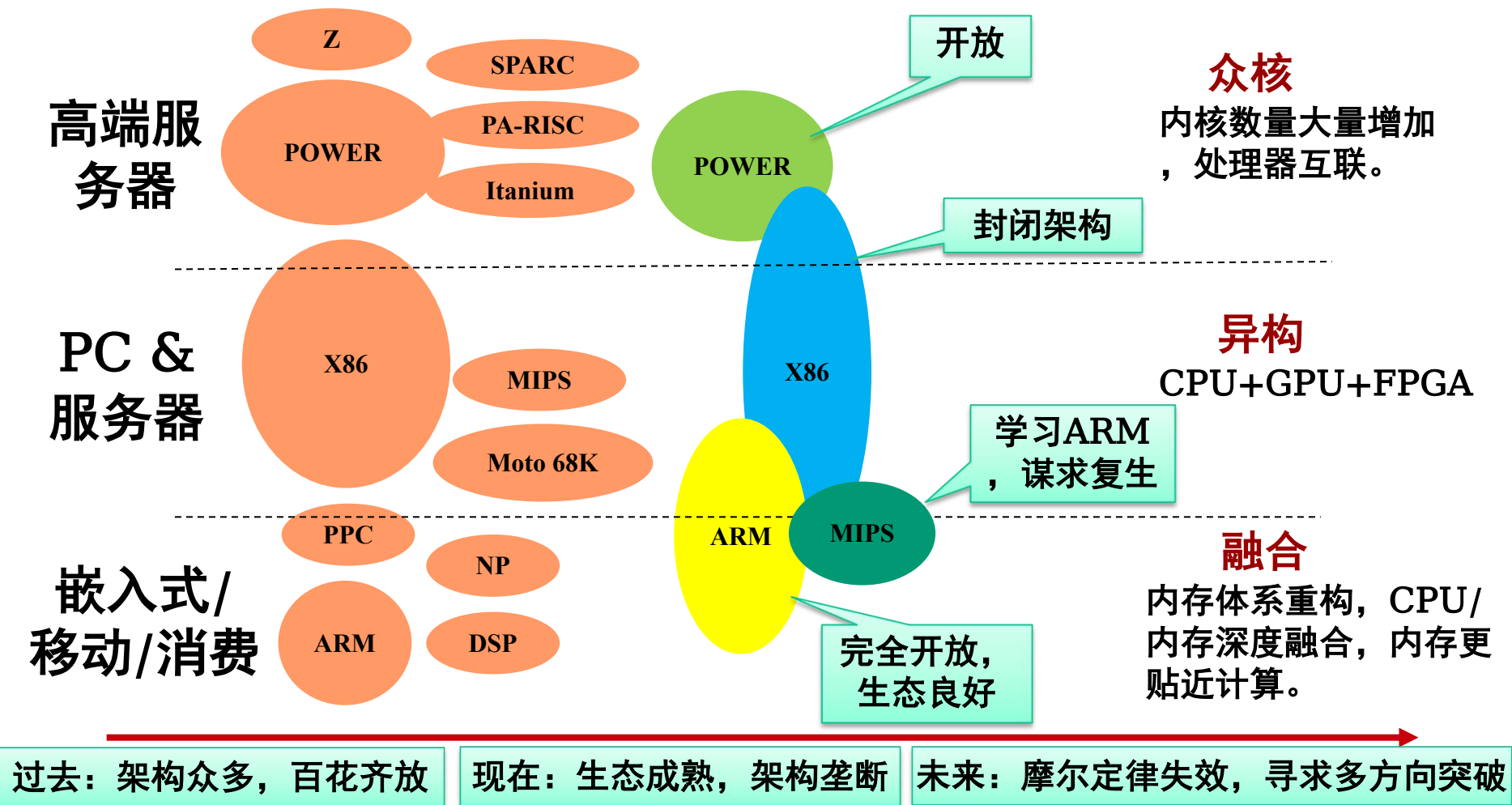


- 功能
 - 解释计算机指令以及处理计算机软件中的数据。
- 组成
 - 控制器，运算器，高速缓冲存储器（Cache）、总线
- 主要产品
 - X86, MIPS, ARM

主流处理器发展



处理器发展



ARM (Advanced RISC Machines) 处理器



- 由ARM公司设计的处理器核心。
- ARM公司通过知识产权 (Intellectually Property, IP) 许可授权转让设计许可, 由与之商业合作的公司开发生产芯片。
 - 华为鲲鹏 (Kunpeng) 920处理器。
- 基于ARM处理器核心生产各具特色的芯片广泛应用
 - 移动通信、便携计算、多媒体数字消费...

ARM 与X86CPU的差异



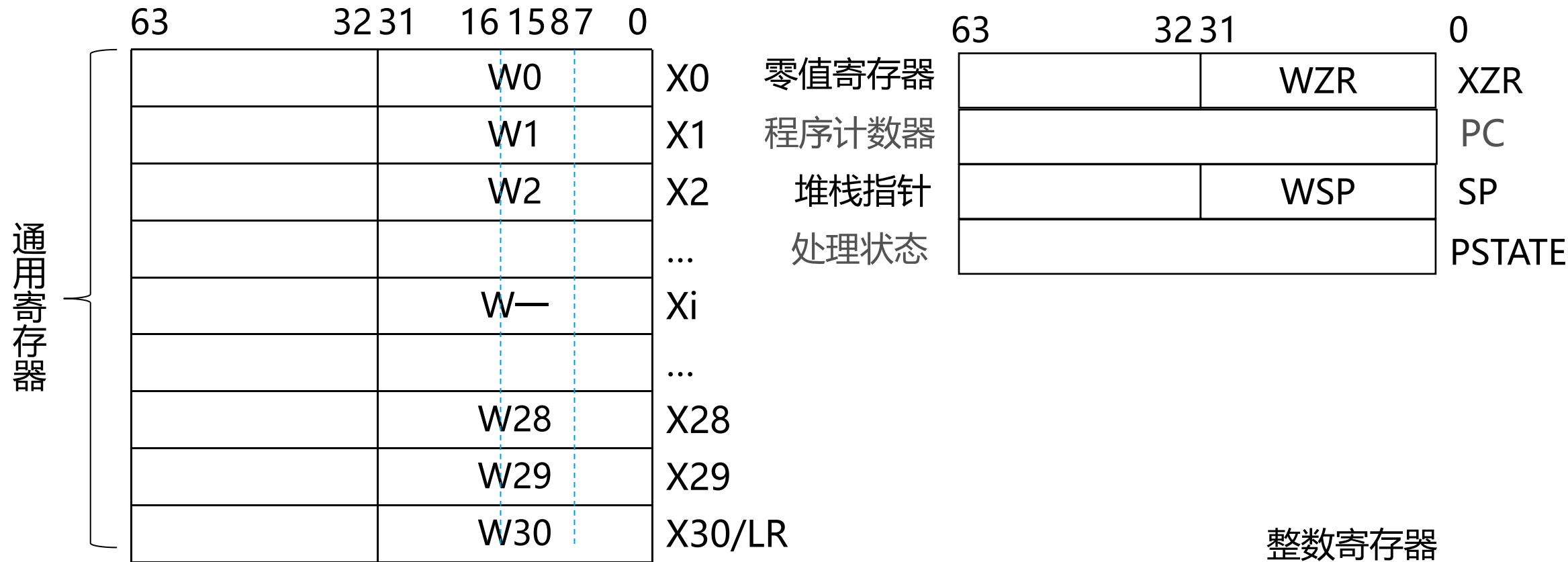
1. char数据类型的符号 — signed
 2. 字节对齐
 3. 嵌入式汇编
 4. SIMD/NEON kernel_fpu_begin/end
 5. 加速引擎 – RDE
 6. 延时
 7. 寄存器集合
 8. 状态寄存器
 9. 指令集
- 指令助记符
 - 指令格式
 - 指令长度
 - 程序格式
 - 伪指令

ARMv8的执行状态

- ARMv8支持两种执行状态（工作模式）
 - AArch64/ARM64： 64位执行状态
 - AArch32： 32位执行状态

	AArch64	AArch32
地址和指令	64位寄存器	32位寄存器
指令集	A64	A32和T32
指令编码	32位定长指令	32位定长指令
兼容性		向后兼容ARMv7（和更早）的代码

AArch64的主要寄存器



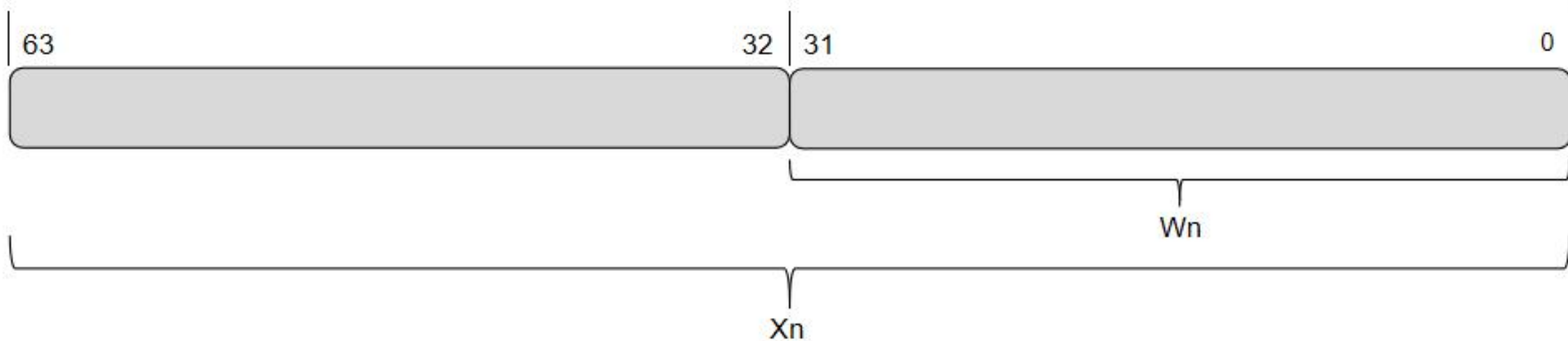
32个SIMD和浮点寄存器, 128位, V0 ~ V31

系统寄存器

64 位与 32 位寄存器的关系



- 64 位寄存器 X_n 的低 32 位为 32 位寄存器 W_n (如 X_0 的低 32 位 = W_0 , X_1 的低 32 位 = W_1 , ..., X_{30} 的低 32 位 = W_{30})
- 操作特性:
 - 对 W_n 操作仅影响低 32 位, 高 32 位自动清零
 - 对 X_n 操作影响整个 64 位



通用寄存器分类 (按功能)

- 参数与返回值寄存器 (X0-X8)
- 临时寄存器 (X9-X17)
- 平台专用寄存器 (X18)
- 被调用者保存寄存器 (X19-X28)
- 特殊功能寄存器 (X29-X30)

常用的专用寄存器



- 为特定功能设计的寄存器，用于控制处理器运行、维护程序状态、支持异常处理等核心操作
- 与通用寄存器的区别：功能固定，通常不用于通用数据存储，而是支撑处理器底层机制
- 保障程序执行顺序、异常处理、权限控制等底层功能的实现

零寄存器	WZR/XZR	作为源寄存器提供常量 0
程序计数器	PC	指向当前指令的下一条指令地址
栈指针寄存器	SP	堆栈栈顶地址
程序状态寄存器	SPSR	保存当前处理器状态 (PSTATE) - 标志位 (N/Z/C/V 等) - D4 位：异常发生时的执行状态 (0=AArch64) - M [3:0]：表示异常发生时的模式或异常级别

ARMv8-A数据存储



- ARMv8-A体系结构既支持小端方式、也支持大端方式。
 - 小端方式存储
 - 4字节的A64指令字采用小端方式存储
 - 所有存储器映射的外设采用小端方式存储
 - 多数ARM处理器默认采用小端方式
 - Linux操作系统默认ARM处理器采用小端方式。
- 存储字长
 - 32位 (4个字节) 为一个字 (Word)
 - 16位 (2个字节) 为半字 (Halfword)
 - 64位 (8个字节) 为双字 (Doubleword)
 - 128位 (16个字节) 为4字 (Quadword)

ARMv8-A存储器组织

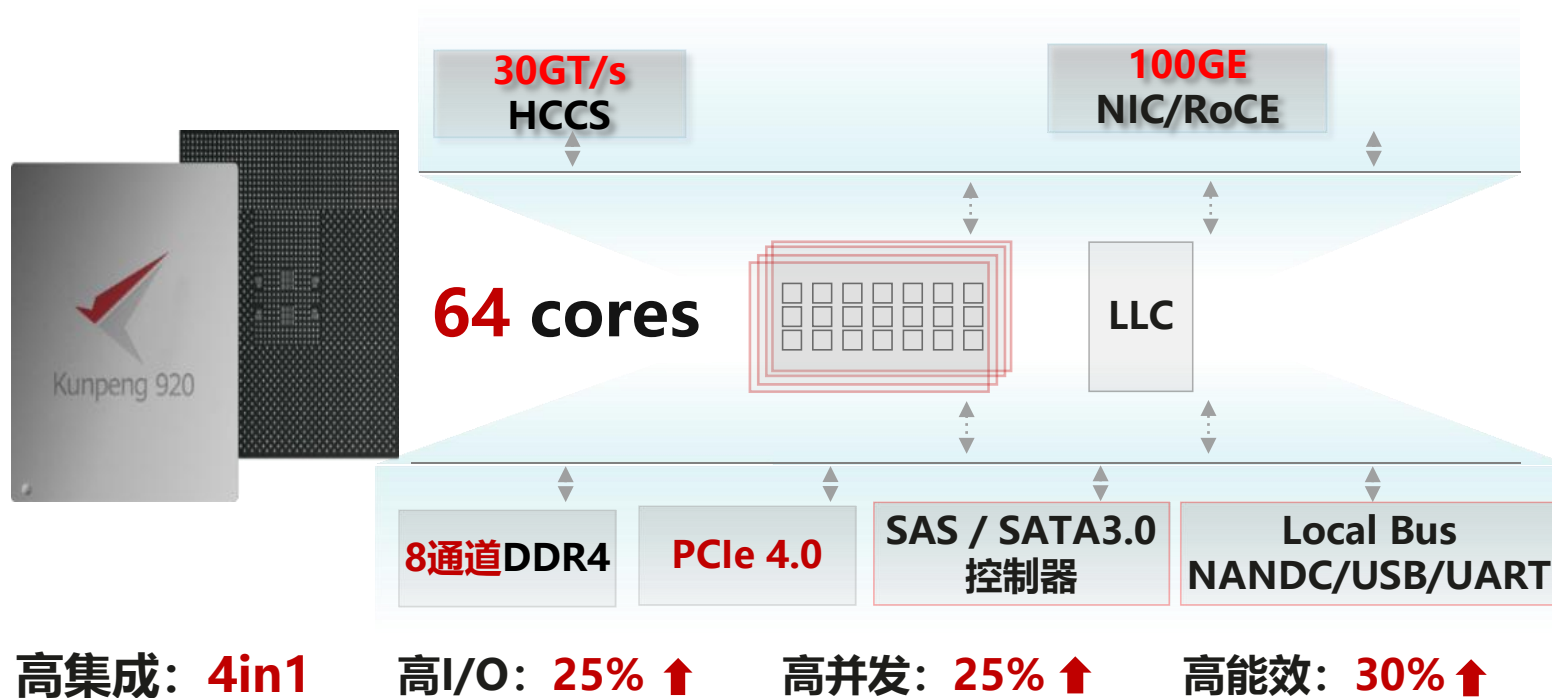


- ARMv8-A实现虚拟存储系统
 - 逻辑地址也就是虚拟地址。
- A64指令集支持64位虚拟地址。
 - 对应用程序来说，面对的都是一样的虚拟存储空间
- 地址边界对齐要求
 - **A64指令必须字（4字节）对齐**
 - 外设地址必须对齐地址
 - 存储器的一般数据访问没有要求必须对齐
 - 有些数据访问则要求地址边界对齐
 - 不符合要求
 - 导致未对齐失效

鲲鹏处理器

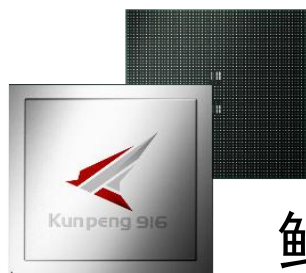


- 业界最高性能ARM-Based处理器
 - 世界首颗64核ARM CPU——华为海思设计
 - 业界性能最高、RISC指令集、7nm工艺



CPU、南桥、网卡、SAS存储控制器等4颗芯片的功能

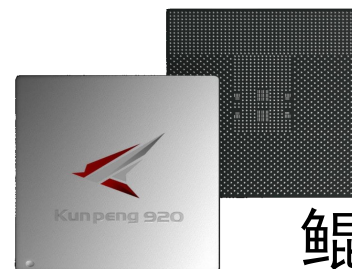
基于ARMv8的鲲鹏处理器



鲲鹏916

支持多路互联的ARM处理器

- ❑ 32核，2.4 GHz主频
- ❑ SPECint性能匹配业界中端，功耗低至75 W
- ❑ 支持4通道DDR4控制器
- ❑ 支持PCIe 3.0和SAS/SATA 3.0
- ❑ 集成板载GE/10 GE网络
- ❑ 支持2路互联



鲲鹏920

7nm制程，数据中心ARM处理器

- ❑ 计算核数提升1倍，最多64核
- ❑ SPECint性能提升超过2倍
- ❑ 内存通道数提升1倍，支持8通道DDR4控制器
- ❑ 支持PCIe 4.0和CCIX
- ❑ 集成板载100 GE网络和加密、压缩等引擎
- ❑ 支持2路或4路互联

超级 CPU 单元



- 每个超级 CPU 单元包含 6 ~ 8 个 CPU 簇 (Cluster)
- 每个 CPU 簇组成:
 - 4 个泰山核 (Core)
 - 3 级 Cache 标签存储器
- 泰山核组成:
 - ARM 核 (基于 ARMv8 体系结构, 完全知识产权)
 - 分离结构的 1 级 Cache
 - 共享结构的 2 级 Cache
- 核心参数: 最多 64 个泰山核, 主频最高 2.6GHz, 仅支持 64 位执行状态

基于鲲鹏主板的多样化计算产品



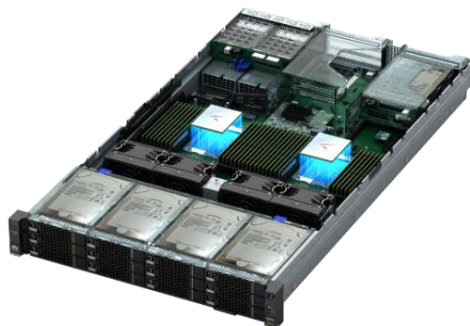
鲲鹏主板开放 共享接口与设备规范



- 主板(含鲲鹏处理器)
- 主板接口规范
- BIOS软件 & 规范
- BMC芯片/软件 & 设备管理规范



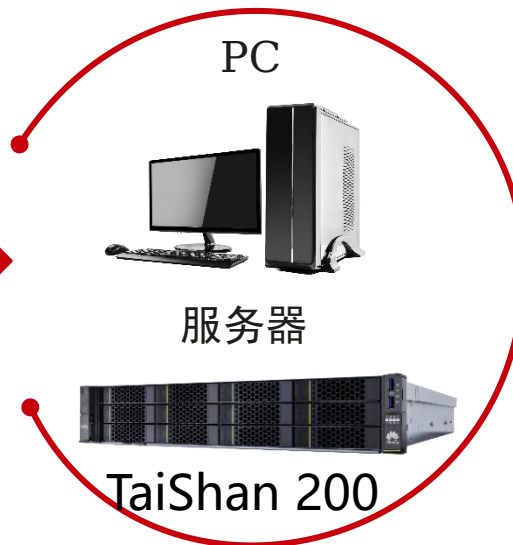
整机参考设计 共享工程能力



- 整机参考设计指南
- (含机箱、散热、供电等设计)
- 部件与OS兼容性列表



打造多样化计算 产品



伙伴价值

持续供应

快速开发
(3个月)

安全可靠
(低于业界平均故障率
15%)

鲲鹏处理器架构特性



- ARM V8.0架构，支持EL0~EL3， 支持Trust-Zone(AArch64 only)
- 兼容ARM V8.1
 - 支持Atomic指令
 - 支持PSTATE.PAN
 - 支持Virtualization Host Extension
- 兼容ARM V8.2
 - 支持半精度浮点指令
 - 支持Dot Product指令
 - 支持RAS Extension
 - 支持Statistical Profiling Extension
- 支持ARM V8.3部分指令：
 - 支持SIMD complex number
 - 支持JavaScript conversion instruction
- 支持ARM V8.4 Memory Partitioning and M Extension (MPAM)
- 支持ARM V8.5 Restrictions on the effects of speculation
 - Support Mitigation for 变种1/2/3/3a/4

鲲鹏处理器流水线技术

- Branch预测和取指流水线解耦设计，取指流水线每拍最多可提供32Bytes指令供译码，分支预测流水线可以不受取指流水停顿影响，超前进行预测处理
- 定浮点流水线分开设计，解除定浮点相互反压，每拍可为后端执行部件提供4条整型微指令及3条浮点微指令
- 整型运算单元支持每拍4条ALU运算（含2条跳转）及1条乘除运算
- 浮点及SIMD运算单元支持每拍2条ARM Neon 128bits 浮点及SIMD运算
- 访存单元支持每拍2条读或写访存操作，读操作最快4拍完成，每拍访存带宽为2x128bits读及1x128bits写



Aarch64汇编语言

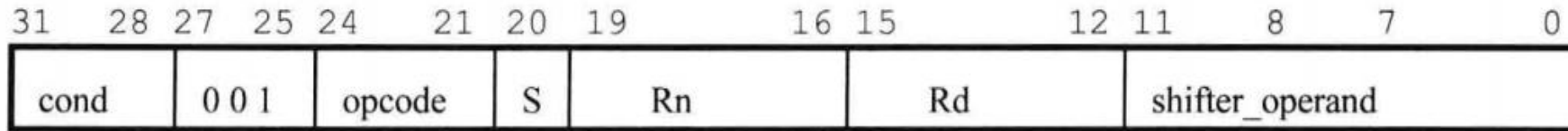
复杂指令集计算机 (Complex Instruction Set Computer)

- ✓ 功能强大而复杂、指令条数繁多
- ✓ 寻址方式多样、指令编码长短不一.....
- ✓ x86体系结构

精简指令集计算机 (Reduced Instruction Set Computer)

- ✓ 固定长度的机器代码、简单的寻址方式
- ✓ 较多的通用寄存器、面向寄存器的数据处理.....
- ✓ MIPS主机 (每秒执行更多百万条指令), ARM

ARM机器指令的编码格式



- opcode: 指令操作编码
- cond: 指令执行的条件编码
- S: 决定指令的操作是否影响CPSR的值
- Rd: 目标寄存器编码
- Rn: 包含第1个操作数的寄存器编码
- shifter operand: 表示第2个操作数

ARM汇编指令的语法格式

- `<opcode>{<cond>}{S}<Rd>,<Rn>,<shifter_operand>`
 - `<opcode>`: 指令助记符, 如ADD
 - `{<cond>}`: 指令执行的条件
 - 在ARMv5之前的版本中, 所有指令都是有条件执行的
 - ARMv5开始, 引入无条件执行的指令
 - 在ARMv8的A64中减少了条件执行指令的数量
 - `{S}`: 决定指令的操作是否影响CPSR
 - `<Rd>`: 表示目标操作数寄存器
 - `<Rn>`: 表示包含第1个操作数的寄存器
 - `<shifter_operand>`: 表示第2个操作数

ARM64的寻址



- A64指令的目的操作数采用寄存器寻址
 - Rd, 是31个64 (和32) 位通用寄存器之一
- 大多数指令的源操作数之一也是寄存器寻址
 - Rn, 是31个64 (和32) 位通用寄存器之一
 - 堆栈指针SP (WSP), 零值寄存器XZR (WZR)
- 第2个源操作数operand2有多种寻址方式
 - 立即数寻址、寄存器寻址Rm, 存储器寻址

ARM64的寄存器寻址



- **寄存器移位寻址**

- operand2形式是: $Rm\{\text{shift imm6}\}$
 - shift: 逻辑左移LSL, 逻辑右移LSR, 算术右移ASR, 循环右移ROR
 - “imm6”: 移位次数
 - 对64位操作是0 ~ 63, 32位操作是0 ~ 31。

- **寄存器扩展寻址**

- operand2形式是: $Rm\{\text{extend imm3}\}$.
 - extend: 对寄存器Rm扩展, 扩展后还可以左移imm3位(0 ~ 4)
 - 无符号数的零位扩展
 - UXTB和UXTH分别把Wm的低字节和低半字数据进行零位扩展
 - 有符号数的符号扩展:
 - SXTB、SXTH和SXTW依次把Wm的低字节、低半字和字数据进行符号扩展

Linux ARM64 ABI (Application Binary Interface, 应用二进制接口) 是保证不同编译器和工具生成的二进制代码能协同运行的核心规范。

- X0-X7 保存函数调用的前8个参数, 超过 8 个的参数通过栈来传递
- 返回值通过寄存器返回X0
- SP按16字节对齐
- 调用者保存寄存器X0-X18, 被调用者保存X19-X29
- 局部变量优先存储在寄存器, 也可以保存到堆栈



鲲鹏处理器汇编示例

eg0102.cpp



```
float f=-1;
double d = -1;
int x = -1;
unsigned int u = 2147483648 ;
long l =-1;
unsigned long ul = -1;
short s =-1;
unsigned short us = -1;
char c='A';
unsigned char uc=200;
char str[]="I Love China!";
char *ps=str;
struct X{
    int a;
    char c;
    int b;
}xx={-100,'a',-200};
```

```
typedef unsigned char *byte_pointer;
void show_bytes(byte_pointer start, size_t len) {
    size_t i;
    for (i = 0; i < len; i++)
        printf(" %.2x", start[i]);
    printf("\n");
}

int main(){
    show_bytes((byte_pointer) &f,sizeof(f));
    show_bytes((byte_pointer) &d,sizeof(d));
    show_bytes((byte_pointer) &x,sizeof(x));
    show_bytes((byte_pointer) &u,sizeof(u));
    show_bytes((byte_pointer) &l,sizeof(l));
    show_bytes((byte_pointer) &ul,sizeof(ul));
    show_bytes((byte_pointer) &s,sizeof(s));
    show_bytes((byte_pointer) &us,sizeof(us));
    show_bytes((byte_pointer) &c,sizeof(c));
    show_bytes((byte_pointer) &uc,sizeof(uc));
    show_bytes((byte_pointer) &str,sizeof(str));
    show_bytes((byte_pointer) &ps,sizeof(ps));
    show_bytes((byte_pointer) &xx,sizeof(xx));
    return 0;
}
```

汇编连接运行



```
g++ eg0102.cpp -S -o eg0102.s
```

```
g++ eg0102.s -g -o eg0102.out
```

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 53 bf aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

eg0102.s(数据定义)



```
float f=-1;
double d = -1;
int x = -1;
unsigned int u = 2147483648 ;
long l =-1;
unsigned long ul = -1;
short s =-1;
unsigned short us = -1;
char c='A';
unsigned char uc=200;
char str[]="I Love China!";
char *ps=str;
struct X{
    int a;
    char c;
    int b;
}xx={-100,'a',-200};
```

```
.data
f:      .align 2
        .word -1082130432
d:      .align 3
        .word 0
        .word -1074790400
x:      .align 2
        .word -1
u:      .word -2147483648
        .align 3
l:      .xword -1
        .align 3
ul:     .xword -1
        .align 1
s:      .hword -1
```

```
us:      .align 1
        .hword -1
c:      .byte 65
uc:     .byte -56
        .align 3
str:     .string "I Love China! "
        .align 3
ps:      .xword str
xx:      .align 3
        .word -100
        .byte 97
        .zero 3
        .word -200
```

伪指令

常用数据定义伪指令



伪指令助记符	说明	示例
.word	4 字节整数	f: .word -1082130432
.xword	8 字节整数	l: .xword -1
.hword	2 字节整数	s: .hword -1
.byte	1 字节整数	c: .byte 65
.string	以 null 结尾的字符串	str: .string "I Love China! "
.zero n	填充 n 个零字节	.zero 3
.align n	当前位置对齐到 2^n 字节边界	.align 2, .align 3

```
float f=-1;      .align 2
```

下一个数据地址按4 字节对齐，是 4 的倍数。

```
f: .word -1082130432
```

分配4 字节整数，初始值为 -1082130432 (0xBF800000)，
浮点数-1.0 的IEEE 754 单精度表示。

```
show_bytes((byte_pointer) &f,sizeof(f));
```

小端方式存储：低字节在低地址

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 53 bf aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
double d = -1;      .align 3  
d: .word 0  
   .word -1074790400
```

下一个数据地址按8 字节对齐，是 8 的倍数。

分配两个4 字节整数，初始值分别为0和-1074790400
(0xBFF0000000000000) ，即浮点数-1.0 的IEEE 754
双精度表示。

```
show_bytes((byte_pointer) &d ,sizeof(d));
```

小端方式存储：低字节在低地址

```
root@flexusx-bcd5:~/assembly# ./eg0102.out  
00 00 80 bf  
00 00 00 00 00 00 f0 bf  
ff ff ff ff  
00 00 00 80  
ff ff ff ff ff ff ff ff  
ff ff ff ff ff ff ff ff  
ff ff  
ff ff  
41  
c8  
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00  
40 10 53 bf aa aa 00 00  
9c ff ff ff 61 00 00 00 38 ff ff ff
```

.align 2 下一个数据地址按4 字节对齐，是 4 的倍数。

int x = -1;

x: .word -1 分配4 字节整数，初始值为 -1 (0xFFFFFFFF) ,
有符号数-1 的补码表示

show_bytes((byte_pointer) &x,sizeof(x));

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 53 bf aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
unsigned int u = 2147483648 ;
```

```
u: .word  -2147483648
```

分配4 字节整数，初始值为 0x80000000，有符号数-2147483648 的补码表示, 无符号数 2147483648 的二进制表示

```
show_bytes((byte_pointer) &u ,sizeof(u));
```

小端方式存储：低字节在低地址

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 53 bf aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
long l = -1;
        .align    3
l: .xword  -1
```

下一个数据地址按8 字节对齐，是 8 的倍数。

分配8 字节整数，初始值为 0xFFFFFFFFFFFFFFFF，
有符号数-1 的补码表示

```
show_bytes((byte_pointer) &l ,sizeof(l));
```

小端方式存储：低字节在低地址

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 53 bf aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

unsigned long ul = -1;	.align 3	下一个数据地址按8 字节对齐, 是 8 的倍数。
ul:	.xword -1	分配8 字节整数, 初始值为 0xFFFFFFFFFFFFFFFF, 有符号数-1 的补码表示

show_bytes((byte_pointer) &ul, sizeof(ul));

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 53 bf aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
short s = -1;      .align 1
```

下一个数据地址按2 字节对齐，是2 的倍数。

```
s: .hword -1
```

分配2 字节整数，初始值为 0xFFFF，有符号数-1 的补码表示

```
show_bytes((byte_pointer) &s ,sizeof(s));
```

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 69 cb aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
unsigned short us = -1; .align 1  
us: .hword -1
```

下一个数据地址按2 字节对齐，是 2 的倍数。
分配2 字节整数，初始值为 0xFFFF，有符号数-1 的补码表示

```
show_bytes((byte_pointer) &us ,sizeof(us));
```

```
root@flexusx-bcd5:~/assembly# ./eg0102.out  
00 00 80 bf  
00 00 00 00 00 00 f0 bf  
ff ff ff ff  
00 00 00 80  
ff ff ff ff ff ff ff ff  
ff ff  
ff ff  
41  
c8  
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00  
40 10 69 cb aa aa 00 00  
9c ff ff ff 61 00 00 00 38 ff ff ff
```



char c= 'A' ; c: .byte 65 分配1 字节整数, 初始值为 65 (0x41) , 字符A的ASCII码

show_bytes((byte_pointer) &c ,sizeof(c));

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 69 cb aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
unsigned char uc=200;  uc: .byte -56
```

分配1 字节整数，初始值为 -56的补码表示（0xc8），
无符号数200的二进制表示

```
show_bytes((byte_pointer) &uc ,sizeof(uc));
```

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 69 cb aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
char str[]="I Love China!";
```

```
str: .string "I Love China! "
```

为以0结尾的字符串每个字符分配一个字节

按从左到右的顺序存储
每个字符的ASCII码

结束字符

```
show_bytes((byte_pointer) &str,sizeof(str));
```

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 69 cb aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

下一个数据地址按8 字节对齐，是 8 的倍数。

```
char *ps=str;      .align      3
ps:                .xword      str
```

分配8 字节整数，初始值为标签str表示的地址

小端方式存储：0x00aaaacb691040

```
show_bytes((byte_pointer) &ps,sizeof(ps));
```

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 69 cb aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

```
struct X{
    int a;
    char c;
    int b;
}xx={-100,'a',-200};
```

```
show_bytes((byte_pointer) &xx ,sizeof(xx));
```

.align 3 下一个数据地址按8 字节对齐,
是 8 的倍数。

xx: .word -100 分配4字节整数, 初始值为-100
.byte 97 分配1字节整数, 值为 'a' 的ASCII码
.zero 3 分配3个零字节
.word -200 分配4字节整数, 初始值为-200

保证地址对齐

```
root@flexusx-bcd5:~/assembly# ./eg0102.out
00 00 80 bf
00 00 00 00 00 00 f0 bf
ff ff ff ff
00 00 00 80
ff ff ff ff ff ff ff ff
ff ff
ff ff
41
c8
49 20 4c 6f 76 65 20 43 68 69 6e 61 21 00
40 10 69 cb aa aa 00 00
9c ff ff ff 61 00 00 00 38 ff ff ff
```

小端方式存储

```
#include <iostream>
int add(int x, int y){
int t;
t = x+y ;
return t;
}
```

```
int main(){
    printf("%d\n", add(100,256));
    return 0;
}
```

```
g++ eg0101.cpp -o eg0101.out
```

```
root@flexusx-bcd5:~/assembly# g++ eg0101.S -o eg0101S.out
root@flexusx-bcd5:~/assembly# ./eg0101S.out
356
```

核心指令

g++ eg0101.cpp -S -o eg0101.s



只读数据段

```
.section .rodata
.LC0:
.string "%d\n "
```

代码段

```
.text
_Z3addii:
    sub    sp, sp, #32
    str    w0, [sp, 12]
    str    w1, [sp, 8]
    ldr    w1, [sp, 12]
    ldr    w0, [sp, 8]
    add    w0, w1, w0
    str    w0, [sp, 28]
    ldr    w0, [sp, 28]
    add    sp, sp, 32
    ret
```

子程序开始

子程序返回

可被外部模块调用

```
.global main
```

程序入口

```
main:
```

//调用C函数的汇编子程序应符合ARM64 ABI规范

```
    stp    x29, x30, [sp, -16]! //分配栈空间, 保存帧指针和返回地址
    mov    w1, 256
    mov    w0, 100
    bl     _Z3addii
    mov    w1, w0
    adrp    x0, .LC0
    add     x0, x0, :lo12:LC0
    bl     printf
    mov    w0, 0
    ldp     x29, x30, [sp], 16 //恢复帧指针和返回地址, 释放栈空间
    ret
```

调试程序



```
root@flexusx-bcd5:~/assembly# apt install gdb
```

```
root@flexusx-bcd5:~/assembly# gdb eg0101.out
```

```
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from eg0101.out...
```



```
(gdb) b main
Breakpoint 1 at 0x8fc: file eg0101.s, line 38.
```

设置断点

```
(gdb) r
```

执行到断点

```
Starting program: /root/assembly/eg0101.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".
```

```
Breakpoint 1, main () at eg0101.s:38
```

主函数的机器指令

```
38      stp     x29, x30, [sp, #-16]!
```

反汇编

```
(gdb) disas /r
```

主函数的汇编指令

```
Dump of assembler code for function main:
```

```
=> 0x0000aaaaaaaa08fc <+0>:
0x0000aaaaaaaa0900 <+4>:
0x0000aaaaaaaa0904 <+8>:
0x0000aaaaaaaa0908 <+12>:
0x0000aaaaaaaa090c <+16>:
0x0000aaaaaaaa0910 <+20>:
0x0000aaaaaaaa0914 <+24>:
0x0000aaaaaaaa0918 <+28>:
0x0000aaaaaaaa091c <+32>:
0x0000aaaaaaaa0920 <+36>:
0x0000aaaaaaaa0924 <+40>:
0x0000aaaaaaaa0928 <+44>:
```

```
fd 7b bf a9
fd 03 00 91
01 20 80 52
80 0c 80 52
f2 ff ff 97
e1 03 00 2a
00 00 00 90
00 20 27 91
99 ff ff 97
00 00 80 52
fd 7b c1 a8
c0 03 5f d6
```

```
stp     x29, x30, [sp, #-16]!
mov     x29, sp
mov     w1, #0x100
mov     w0, #0x64
bl      0xaaaaaaaa08d4 <_Z3addii>
mov     w1, w0
adrp    x0, 0xaaaaaaaa0000
add     x0, x0, #0x9c8
bl      0xaaaaaaaa0780 <printf@plt>
mov     w0, #0x0
ldp     x29, x30, [sp], #16
ret
```

```
// #256
// #100
// #0
```

```
End of assembler dump.
```

- ARM64 指令长度固定4字节
 - RISC指令

主函数的机器指令

A64 汇编指令的典型格式:

指令助记符{条件} {S} 目的操作数, 源操作数1|SP, {源操作数2}

Dump of assembler code for function `main`:

<code>=> 0x0000aaaaaaaa08fc <+0>:</code>	<code>fd 7b bf a9</code>	<code>stp</code>	<code>x29, x30, [sp, #-16]!</code>
<code>0x0000aaaaaaaa0900 <+4>:</code>	<code>fd 03 00 91</code>	<code>mov</code>	<code>x29, sp</code>
<code>0x0000aaaaaaaa0904 <+8>:</code>	<code>01 20 80 52</code>	<code>mov</code>	<code>w1, #0x100 // #256</code>
<code>0x0000aaaaaaaa0908 <+12>:</code>	<code>80 0c 80 52</code>	<code>mov</code>	<code>w0, #0x64 // #100</code>
<code>0x0000aaaaaaaa090c <+16>:</code>	<code>f2 ff ff 97</code>	<code>bl</code>	<code>0xaaaaaaaa08d4 <_Z3addii></code>
<code>0x0000aaaaaaaa0910 <+20>:</code>	<code>e1 03 00 2a</code>	<code>mov</code>	<code>w1, w0</code>
<code>0x0000aaaaaaaa0914 <+24>:</code>	<code>00 00 00 90</code>	<code>adrp</code>	<code>x0, 0xaaaaaaaa0000</code>
<code>0x0000aaaaaaaa0918 <+28>:</code>	<code>00 20 27 91</code>	<code>add</code>	<code>x0, x0, #0x9c8</code>
<code>0x0000aaaaaaaa091c <+32>:</code>	<code>99 ff ff 97</code>	<code>bl</code>	<code>0xaaaaaaaa0780 <printf@plt></code>
<code>0x0000aaaaaaaa0920 <+36>:</code>	<code>00 00 80 52</code>	<code>mov</code>	<code>w0, #0x0 // #0</code>
<code>0x0000aaaaaaaa0924 <+40>:</code>	<code>fd 7b c1 a8</code>	<code>ldp</code>	<code>x29, x30, [sp], #16</code>
<code>0x0000aaaaaaaa0928 <+44>:</code>	<code>c0 03 5f d6</code>	<code>ret</code>	

End of assembler dump.

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                 // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：将 x29（帧指针）和 x30（链接寄存器，保存返回地址）成对存储到栈上，栈指针向低地址移动。
- 具体操作：
 - sp 先减去 16 字节（#-16），使栈向下增长 16 字节。
 - 然后将 x29 存到新栈顶 [sp]，x30 存到 [sp+8]。
 - ! 表示将更新后的 sp 写回寄存器。
- 作用：根据ARM64ABI规范，保存调用者的帧指针和返回地址，为当前函数建立栈帧，同时保持栈指针 16 字节对齐。

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：将当前栈指针 `sp` 的值复制到 `x29`。
- 作用：让 `x29` 指向当前栈帧的底部（即刚分配的 16 字节栈空间的起始位置），作为栈帧的基准。

MOV Rd, Rs // Rd = Rs

Rd: 寄存器

Rs: 寄存器, 立即数

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:      fd 7b bf a9      stp      x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:      fd 03 00 91      mov      x29, sp
    0x0000aaaaaaaa0904 <+8>:      01 20 80 52      mov      w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:     80 0c 80 52      mov      w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:     f2 ff ff 97      bl       0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:     e1 03 00 2a      mov      w1, w0
    0x0000aaaaaaaa0914 <+24>:     00 00 00 90      adrp     x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:     00 20 27 91      add      x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:     99 ff ff 97      bl       0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:     00 00 80 52      mov      w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:     fd 7b c1 a8      ldp      x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:     c0 03 5f d6      ret
```

- 功能：将立即数 0x100（256）移入 w1（32 位寄存器），立即数 0x64（100）移入 w0
- 作用：设置函数 `add` 的入口参数
 - 按照 ARM64 调用约定，参数依次使用 x0/w0、x1/w1....。

Dump of assembler code for function **main**:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：带链接的分支，跳转到地址 0xaaaaaaaa08d4 处的 `_Z3addii` 函数。
- 具体操作：
 - 将下一条指令的地址（0xaaaaaaaa0910）保存到链接寄存器 x30。
 - 跳转到目标地址。
- 作用：调用 `add(100, 256)` 函数，并将返回地址存入 x30

BL label /* PC+4→X30,PC=PC+位移量 */

Dump of assembler code for function main:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：将 w0 (add 函数的返回值, 即 356) 复制到 w1。
- 作用：将计算结果作为 printf 的第二个参数 (printf ("%d\n" , add(100, 256)))。

Dump of assembler code for function **main**:

```
=> 0x0000aaaaaaaa08fc <+0>:      fd 7b bf a9      stp      x29, x30, [sp, #-16]!
0x0000aaaaaaaa0900 <+4>:      fd 03 00 91      mov      x29, sp
0x0000aaaaaaaa0904 <+8>:      01 20 80 52      mov      w1, #0x100                // #256
0x0000aaaaaaaa0908 <+12>:     80 0c 80 52      mov      w0, #0x64                // #100
0x0000aaaaaaaa090c <+16>:     f2 ff ff 97      bl       0xaaaaaaaa08d4 <_Z3addii>
0x0000aaaaaaaa0910 <+20>:     e1 03 00 2a      mov      w1, w0
0x0000aaaaaaaa0914 <+24>:     00 00 00 90      adrp     x0, 0xaaaaaaaa0000
0x0000aaaaaaaa0918 <+28>:     00 20 27 91      add      x0, x0, #0x9c8
0x0000aaaaaaaa091c <+32>:     99 ff ff 97      bl       0xaaaaaaaa0780 <printf@plt>
0x0000aaaaaaaa0920 <+36>:     00 00 80 52      mov      w0, #0x0                // #0
0x0000aaaaaaaa0924 <+40>:     fd 7b c1 a8      ldp      x29, x30, [sp], #16
0x0000aaaaaaaa0928 <+44>:     c0 03 5f d6      ret
```

- 功能：地址计算，将 0xaaaaaaaa0000 所在页的基地址加上当前 PC 的低 12 位偏移，得到页对齐的地址存入 x0。
- 具体操作：x0 = (0xaaaaaaaa0000 + (PC & ~0xFFF)) 的页面基址。
- 作用：加载格式字符串 ("%d\n") 所在内存页的基址，配合下一条指令形成完整地址。

adrp x1, label → X1 = (PC低12位清零) + (偏移量 << 12)

Dump of assembler code for function main:

```
=> 0x0000aaaaaaaa08fc <+0>:      fd 7b bf a9      stp      x29, x30, [sp, #-16]!
0x0000aaaaaaaa0900 <+4>:      fd 03 00 91      mov      x29, sp
0x0000aaaaaaaa0904 <+8>:      01 20 80 52      mov      w1, #0x100                // #256
0x0000aaaaaaaa0908 <+12>:     80 0c 80 52      mov      w0, #0x64                // #100
0x0000aaaaaaaa090c <+16>:     f2 ff ff 97      bl       0xaaaaaaaa08d4 <_Z3addii>
0x0000aaaaaaaa0910 <+20>:     e1 03 00 2a      mov      w1, w0
0x0000aaaaaaaa0914 <+24>:     00 00 00 90      adrp     x0, 0xaaaaaaaa0000
0x0000aaaaaaaa0918 <+28>:     00 20 27 91      add      x0, x0, #0x9c8
0x0000aaaaaaaa091c <+32>:     99 ff ff 97      bl       0xaaaaaaaa0780 <printf@plt>
0x0000aaaaaaaa0920 <+36>:     00 00 80 52      mov      w0, #0x0                // #0
0x0000aaaaaaaa0924 <+40>:     fd 7b c1 a8      ldp      x29, x30, [sp], #16
0x0000aaaaaaaa0928 <+44>:     c0 03 5f d6      ret
```

- 功能：给 x0 加上立即数 0x9c8。
- 结果：x0 最终指向格式字符串的实际地址（即 .LC0 在内存中的位置）。
- 作用：设置 printf 的第一个参数（ printf ("%d\n" , add(100, 256)) ）。

```
(gdb) x/8xb 0xaaaaaaaa09c8
```

```
0xaaaaaaaa09c8: 0x25  0x64  0x0a  0x00  0x01  0x1b  0x03  0x3b
```

ADD{S} Rd, Rs1, Rs2 //加法: $Rd = Rs1 + Rs2$

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：跳转到 `printf` 的 PLT（过程链接表）入口，并将返回地址保存到 `x30`。
- 作用：调用 `printf` 函数，输出结果 `(356\n)`。

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：将立即数 0 移入 w0。
- 作用：设置 main 函数的返回值（0 表示正常退出）。

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:      fd 7b bf a9      stp      x29, x30, [sp, #-16]!
0x0000aaaaaaaa0900 <+4>:      fd 03 00 91      mov      x29, sp
0x0000aaaaaaaa0904 <+8>:      01 20 80 52      mov      w1, #0x100                // #256
0x0000aaaaaaaa0908 <+12>:     80 0c 80 52      mov      w0, #0x64                // #100
0x0000aaaaaaaa090c <+16>:     f2 ff ff 97      bl      0xaaaaaaaa08d4 <_Z3addii>
0x0000aaaaaaaa0910 <+20>:     e1 03 00 2a      mov      w1, w0
0x0000aaaaaaaa0914 <+24>:     00 00 00 90      adrp     x0, 0xaaaaaaaa0000
0x0000aaaaaaaa0918 <+28>:     00 20 27 91      add      x0, x0, #0x9c8
0x0000aaaaaaaa091c <+32>:     99 ff ff 97      bl      0xaaaaaaaa0780 <printf@plt>
0x0000aaaaaaaa0920 <+36>:     00 00 80 52      mov      w0, #0x0                // #0
0x0000aaaaaaaa0924 <+40>:     fd 7b c1 a8      ldp      x29, x30, [sp], #16
0x0000aaaaaaaa0928 <+44>:     c0 03 5f d6      ret
```

- 功能：成对加载，从栈中恢复 x29 和 x30，并后增栈指针。
- 具体操作：
 - 从当前 sp 处加载 16 字节数据：x29 ← [sp], x30 ← [sp+8]。
 - 然后 sp 增加 16 字节 ([sp], 16)。
- 作用：恢复调用者的帧指针和返回地址，释放之前stp指令分配的栈空间。

Dump of assembler code for function `main`:

```
=> 0x0000aaaaaaaa08fc <+0>:    fd 7b bf a9    stp     x29, x30, [sp, #-16]!
    0x0000aaaaaaaa0900 <+4>:    fd 03 00 91    mov     x29, sp
    0x0000aaaaaaaa0904 <+8>:    01 20 80 52    mov     w1, #0x100                // #256
    0x0000aaaaaaaa0908 <+12>:   80 0c 80 52    mov     w0, #0x64                // #100
    0x0000aaaaaaaa090c <+16>:   f2 ff ff 97    bl      0xaaaaaaaa08d4 <_Z3addii>
    0x0000aaaaaaaa0910 <+20>:   e1 03 00 2a    mov     w1, w0
    0x0000aaaaaaaa0914 <+24>:   00 00 00 90    adrp    x0, 0xaaaaaaaa0000
    0x0000aaaaaaaa0918 <+28>:   00 20 27 91    add     x0, x0, #0x9c8
    0x0000aaaaaaaa091c <+32>:   99 ff ff 97    bl      0xaaaaaaaa0780 <printf@plt>
    0x0000aaaaaaaa0920 <+36>:   00 00 80 52    mov     w0, #0x0                // #0
    0x0000aaaaaaaa0924 <+40>:   fd 7b c1 a8    ldp     x29, x30, [sp], #16
    0x0000aaaaaaaa0928 <+44>:   c0 03 5f d6    ret
```

- 功能：从函数返回。
- 具体操作：跳转到 x30 中保存的地址。
- 作用：返回到调用 main 的 C 运行时（如 `__libc_start_main`），结束程序。

RET {Xn} /* 跳转到Xn(X30)寄存器保存的地址 (PC = Xn) */

设置断点

```
(gdb) b _Z3addii
```

```
Breakpoint 2 at 0xaaaaaaaa08d4: file eg0101.s, line 12.
```

执行到断点

```
(gdb) c
```

```
Continuing.
```

```
Breakpoint 2, add(int, int) () at eg0101.s:12
```

```
12      sub    sp, sp, #32
```

反汇编

```
(gdb) disas /r
```

```
Dump of assembler code for function _Z3addii:
```

add函数的汇编指令

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

```
End of assembler dump.
```

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
End of assembler dump.
```

- 功能：将栈指针 `sp` 减去 32 字节（0x20）。
- 作用：为当前函数在栈上分配 32 字节的局部存储空间，用于保存参数、临时变量等。

SUB{S} Rd, Rs1, Rs2 ; 减法: $Rd = Rs1 - Rs2$

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：将 w0（参数 a）存储到栈地址 [sp + 12] 处，将 w1（参数 b）存储到栈地址 [sp + 8] 处
- 作用：将寄存器中的参数保存到栈上（因为之后会重用 w0, w1 寄存器）。

`str Rs, 内存地址 // 存储 32/64 位数据到内存`

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：从栈地址 `[sp + 12]` 加载数据到 `w1`, 栈地址 `[sp + 8]` 加载数据到 `w0`
- 作用：将之前保存的参数 `a` 重新加载到 `w1`, 参数 `b` 重新加载到 `w0`
- 注意：实际上交换了 `w0` 和 `w1` 的内容（但因为加法满足交换律，不影响结果）。

`ldr Rd, 内存地址` // 加载 32/64 位数据到 `Rd`

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：将 w1 和 w0 相加，结果存入 w0。
- 作用：计算 $a + b$ ，结果放在 w0。

ADD{S} Rd, Rs1, Rs2 //加法: $Rd = Rs1 + Rs2$

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：将 w0 中的结果存储到栈地址 [sp + 28] 处。
- 作用：将加法结果临时保存到栈上。

`str` Rs, 内存地址 // 存储 32/64 位数据到内存

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：从栈地址 `[sp + 28]` 加载数据回 `w0`。
- 作用：将刚才保存的结果重新取回 `w0`，作为函数返回值。

`ldr Rd, 内存地址` `// 加载 32/64 位数据到 Rd`

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub      sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str      w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str      w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr      w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr      w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add      w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str      w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr      w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add      sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：将栈指针 `sp` 加上 32 字节。
- 作用：释放之前分配的栈空间，恢复调用前的栈指针。

ADD{S} Rd, Rs1, Rs2 //加法: $Rd = Rs1 + Rs2$

Dump of assembler code for function `_Z3addii`:

```
=> 0x0000aaaaaaaa08d4 <+0>:      ff 83 00 d1      sub     sp, sp, #0x20
    0x0000aaaaaaaa08d8 <+4>:      e0 0f 00 b9      str     w0, [sp, #12]
    0x0000aaaaaaaa08dc <+8>:      e1 0b 00 b9      str     w1, [sp, #8]
    0x0000aaaaaaaa08e0 <+12>:     e1 0f 40 b9      ldr     w1, [sp, #12]
    0x0000aaaaaaaa08e4 <+16>:     e0 0b 40 b9      ldr     w0, [sp, #8]
    0x0000aaaaaaaa08e8 <+20>:     20 00 00 0b      add     w0, w1, w0
    0x0000aaaaaaaa08ec <+24>:     e0 1f 00 b9      str     w0, [sp, #28]
    0x0000aaaaaaaa08f0 <+28>:     e0 1f 40 b9      ldr     w0, [sp, #28]
    0x0000aaaaaaaa08f4 <+32>:     ff 83 00 91      add     sp, sp, #0x20
    0x0000aaaaaaaa08f8 <+36>:     c0 03 5f d6      ret
```

End of assembler dump.

- 功能：从函数返回。
- 作用：跳转到 x30（链接寄存器）中保存的返回地址，返回到调用函数（如 main）。

`RET {Xn} /* 跳转到Xn(X30)寄存器保存的地址 (PC = Xn) */`

简化程序框架



```
.section .rodata
```

```
//定义常量
```

```
.data
```

```
//定义初始化变量
```

```
.text
```

```
//自定义子程序
```

```
.global main
```

```
main:
```

```
//调用C函数的汇编子程序应符合ARM64 ABI规范
```

```
stp    x29, x30, [sp, -16]! //分配栈空间, 保存帧指针和返回地址
```

```
//自己的代码
```

```
ldp    x29, x30, [sp], 16 //恢复帧指针和返回地址, 释放栈空间
```

```
ret
```

ARM汇编语言程序框架



A64 汇编语言源程序框架 (eg0700.S)

.data // 数据区

/* 数据定义 */

.text /* 代码区 */

.global main

main:

/* 保护寄存器 X29 和 X30 */

stp x29, x30, [sp, -16]!

... /* 汇编语言指令 */

mov x0, 0 /* 返回值 */

/* 恢复寄存器 X29 和 X30 */

ldp x29, x30, [sp], 16

ret /* 主函数返回 */

//子程序

/* AS x86汇编语言基本框架eg0000.S */

.intel_syntax noprefix

.data #定义数据段

.....

.text #定义代码段

.globl main

main:

.....

mov eax, 0

ret

..... #子程序